

**ЧАСТНА ПРОФЕСИОНАЛНА ГИМНАЗИЯ ЗА ДИГИТАЛНИ НАУКИ „СОФТУНИ
БУДИТЕЛ“, гр. София**

ДИПЛОМЕН ПРОЕКТ

на Ангел Тончев Делчев

ученик от XII А. клас

професия- код: 481030, “Приложен програмист”

специалност- код: 4810301, “Приложно програмиране”

**Тема: Уеб базирана платформа за обяви за работа и професионална мрежа
"FindAJob"**

Ръководител-консултант: Лечо Лечев

Сесия: май-юни 2026г.

Дата: 09/05/2026

СЪДЪРЖАНИЕ

1. Увод	4
2. Изложение	4
2.1. Глава 1 Проучвателна част	5
2.2. Глава 2 Изисквания към програмния продукт	5
2.3. Глава 3 Проектен цикъл на приложението	5
2.4. Глава 4 Ръководство за потребителя	5
3. Заключение	6
4. Информационни източници	7
5. Приложения	8

1. Увод

Дипломният проект представя разработка в областта на съвременните уеб базирани информационни системи за подбор на персонал и управление на човешки ресурси. FindAJob е модерно, пълнофункционално уеб базирано приложение от корпоративен клас (enterprise-grade), проектирано да служи като мост между работодателите и търсещите работа. В свят, в който дигиталното присъствие е от решаващо значение за кариерното развитие, FindAJob предлага интуитивен интерфейс, съчетан с мощна сървърна логика, за да улесни процеса на подбор на персонал.

Целеви групи на приложението:

- **Търсещи работа (Кандидати):** Индивиди, които търсят кариерно развитие. Те имат нужда от платформа, където да могат да изградят детайлен професионален профил, да търсят обяви по ключови думи и филтри, да запазват интересни позиции и да кандидатстват максимално бързо.
- **Работодатели:** Компании или HR специалисти, търсещи да привлекат нови таланти. Те разполагат с инструменти за публикуване на детайлни обяви, разглеждане на получени кандидатури, филтриране на кандидати и управление на целия процес по подбор.
- **Администратори:** Лица, отговарящи за поддръжката и модерацията на платформата. Те контролират потребителските акаунти, одобряват нови регистрации на работодатели и модерират съдържанието.

2. Изложение

2.1. Глава 1 Проучвателна част

Преди да започне фазата на проектиране, беше извършено задълбочено изследване на пазара и анализ на утвърдените платформи за търсене и предлагане на работа: Jobs.bg, Indeed и LinkedIn.

Jobs.bg Най-разпознаваемата и масово използвана платформа на територията на България.

- **Силни страни:** Огромна база от потребители и работодатели, силна локализация и навици на местните потребители.
- **Слаби страни:** Остарял и статичен потребителски интерфейс (UI). Липсват съвременни социални функции. Комуникацията е тромава, тъй като често се налага преминаване към външни канали (лични имейли или телефонни обаждания), което води до загуба на контекст.

Indeed Световен агрегатор на обяви за работа.

- **Силни страни:** Мощен алгоритъм за търсене, глобален обхват, способност да извлича обяви от корпоративни сайтове.
- **Слаби страни:** Претрупан и често объркващ интерфейс. Ниско качество на част от обявите (поради автоматичното им изтегляне), както и слаба директна връзка между кандидата и работодателя.

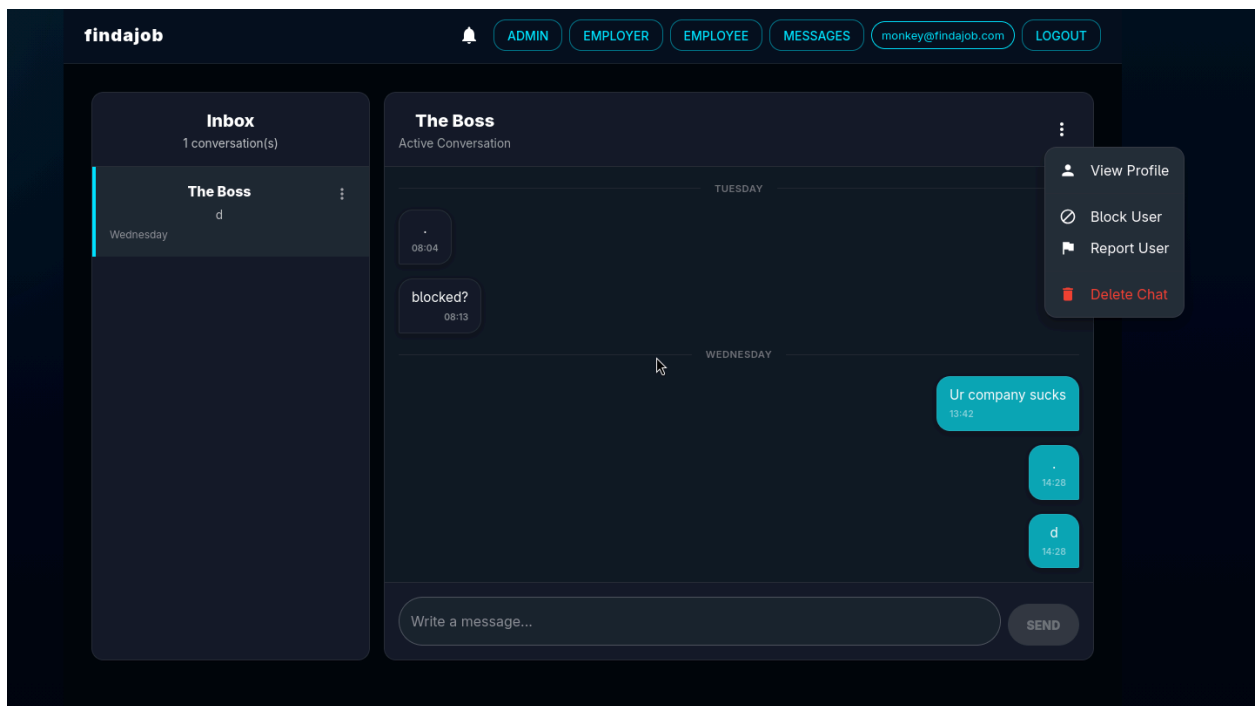
LinkedIn Водещата глобална професионална социална мрежа.

- **Силни страни:** Богати професионални идентичности, "Easy Apply" (Бързо кандидатстване), отлични възможности за нетуъркинг.
- **Слаби страни:** Изключително високи такси за работодателите при публикуване на обяви. Платформата често страда от прекалено много "шум" под формата на непрофесионално или маркетинг съдържание в информационния поток.

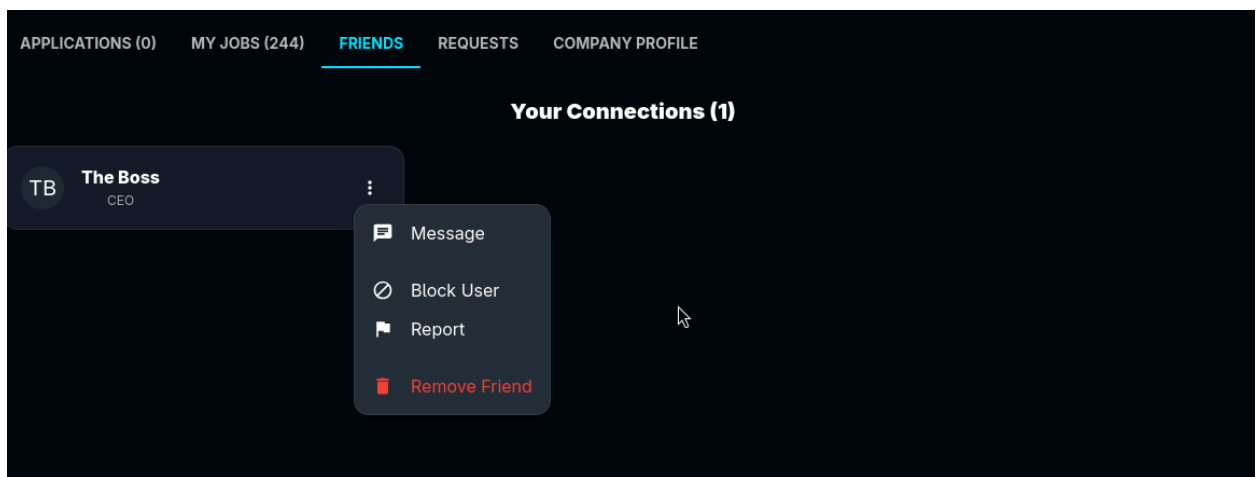
Анализ и изводи: От проведения анализ стана ясно, че традиционните "борси" (Jobs.bg) са прекалено изолирани и статични, докато социалните мрежи (LinkedIn) са прекалено скъпи и разфокусирани. Липсва хибридно решение – платформа, която да съчетава строгостта и фокуса на традиционна борса за работа, с бързата и интерактивна комуникация на съвременните социални приложения.

Въз основа на горепосочения анализ, в **FindAJob** внедрихме следните иновации:

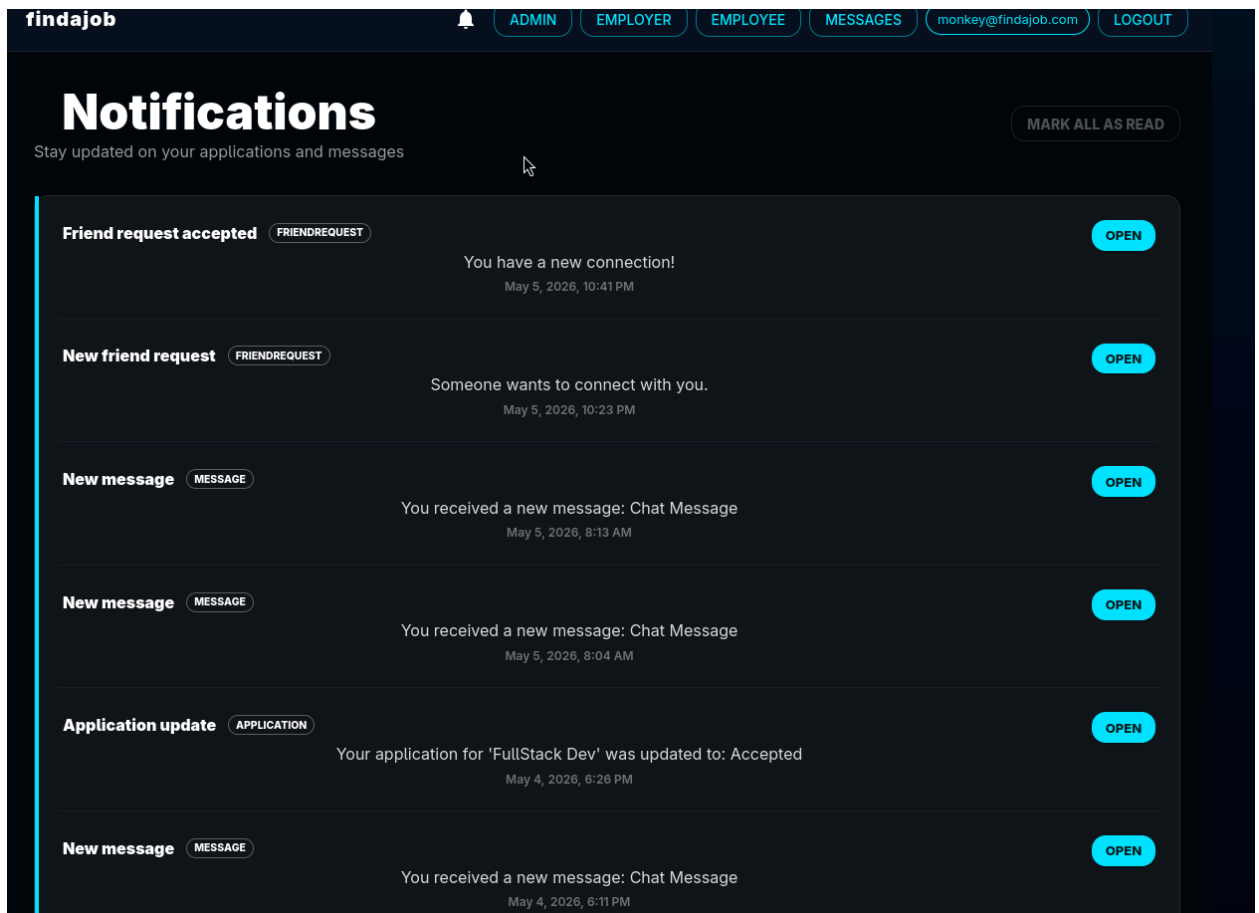
1. **Интегрирана чат система:** Премахва нуждата от преминаване към външни имейл платформи.



2. **Система за професионални приятелства:** Позволява изграждането на доверена мрежа от контакти.



3. Автоматични статус-известия: Потребителят веднага разбира, когато неговата кандидатура е прегледана или статусът ѝ е променен.



2.2. Глава 2 Изисквания към програмния продукт

Проектът трябва да изпълнява следните технически и функционални изисквания:

- **Архитектура:** Клиент-сървър (Client-Server) модел, разделен на самостоятелна уеб-клиентска част (Frontend) и сървърен програмен интерфейс (Backend REST API).
- **Сървърни изисквания:** Изграждане на сървърна част с ASP.NET Core 10.

- **Клиентски изисквания:** Изграждане чрез ReactJS и TypeScript.
- **База данни:** Използване на релационна база данни чрез Entity Framework Core (Code-First) с методи за първоначално пълнене на данни (seeding). Базата данни трябва да съдържа минимум 5 обекта (Entities).
- **Компоненти на приложението:** Поне 5 изгледа (Views), 5 модела (Models) и 5 контролера (Controllers).
- **Известяване и комуникация:** Логика за автоматично уведомяване на кандидатите и работодателите при нови действия; интеграция на чат.
- **Административна част:** Панел за модерация на обяви, управление на потребители и извличане на статистики.
- **Потребителски профили:** Възможност за качване на CV и проследяване на кандидатури.
- **Тестване:** Създаване на Unit тестове за проверка на бизнес логиката (минимум 65% покритие), както и оптимизация на производителността.
- **Разгръщане (Deployment):** CI/CD процеси и задължително деплойване в интернет среда чрез Docker контейнеризация.

2.3. Глава 3 Проектен цикъл на приложението

Избор на технологии

- **Сървърна част (Backend):** Изградена с ASP.NET Core 10.0 Web API. Осигурява бързодействие, сигурност и лесно мащабиране. Използва се моделът на внедряване на зависимости (Dependency Injection).
- **Клиентска част (Frontend):** React 18, написан на TypeScript. Изграден чрез Vite за максимална бързина при компилация. Стилизирането е направено с Vanilla CSS.

- База данни (Database): SQLite (с Entity Framework Core). Подходът Code-First позволява базата да се генерира директно от C# класовете.

Архитектура на базата данни (Entities) Базата данни е проектирана с висока степен на нормализация и включва следните основни обекти:

1. **AspNetUsers** - Разширява стандартния Identity потребител, като съхранява допълнителни данни като **CompanyName** и **ProfessionalTitle**.
2. **JobPostings** (Обяви за работа) - Съхранява заглавие, описание, локация, заплата, тип заетост (Remote/Hybrid) и флаг **IsDeleted**.
3. **JobApplications** (Кандидатури) - Запис за подадена кандидатура със статус и връзка към обявата и потребителя.
4. **UserProfiles** (Профили) - Разширена информация за кандидата (Био, Снимка, Банер).
5. **Messages** (Съобщения) - Записи на всички вътрешносистемни съобщения в реално време.
6. **Notifications** (Известия) - Системни известия за потребителите (напр. "Имате ново съобщение").
7. **CvDocuments** (Автобиографии) - Пътища към качените PDF файлове.
8. **Friendships / FriendRequests** - За управление на професионални мрежи.

Разработка на сървърната част (Backend) Сървърната архитектура е разделена на контролери (API endpoints) и услуги (Service Layer).

Управление на обяви (JobService): Отговаря за извличане, създаване и редактиране на обявите, както и управлението на таговете чрез междинната таблица **JobPostingTags**. Логика за извличане на активни обяви (без изтритите):

```
public async Task<List<JobPosting>> GetJobsAsync()

{

    using var context = await _factory.CreateDbContextAsync();

    return await context.JobPostings

        .Include(j => j.JobPostingTags).ThenInclude(jt => jt.Tag)

        .Where(j => !j.IsDeleted) // Филтриране на "меко" изтритите обяви

        .OrderByDescending(j => j.CreatedAt)

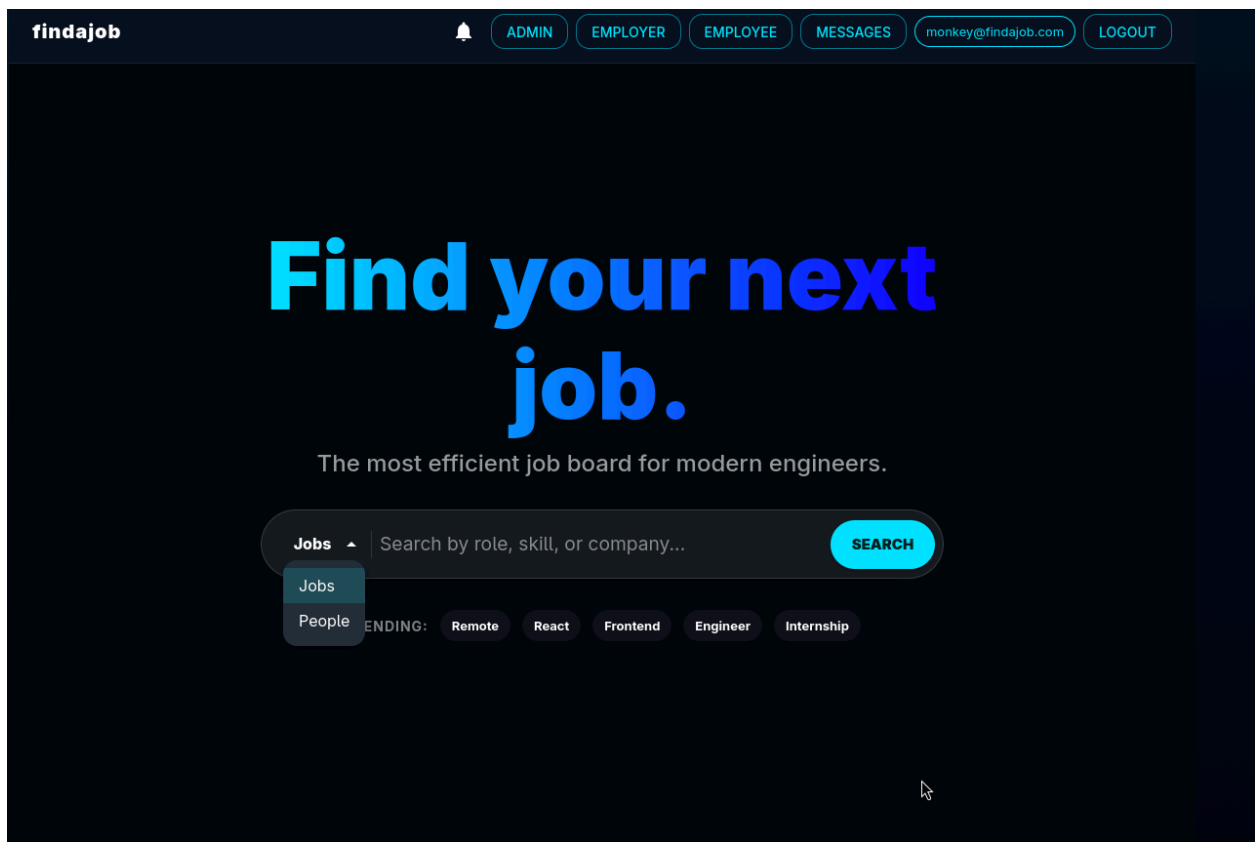
        .ToListAsync();

}
```

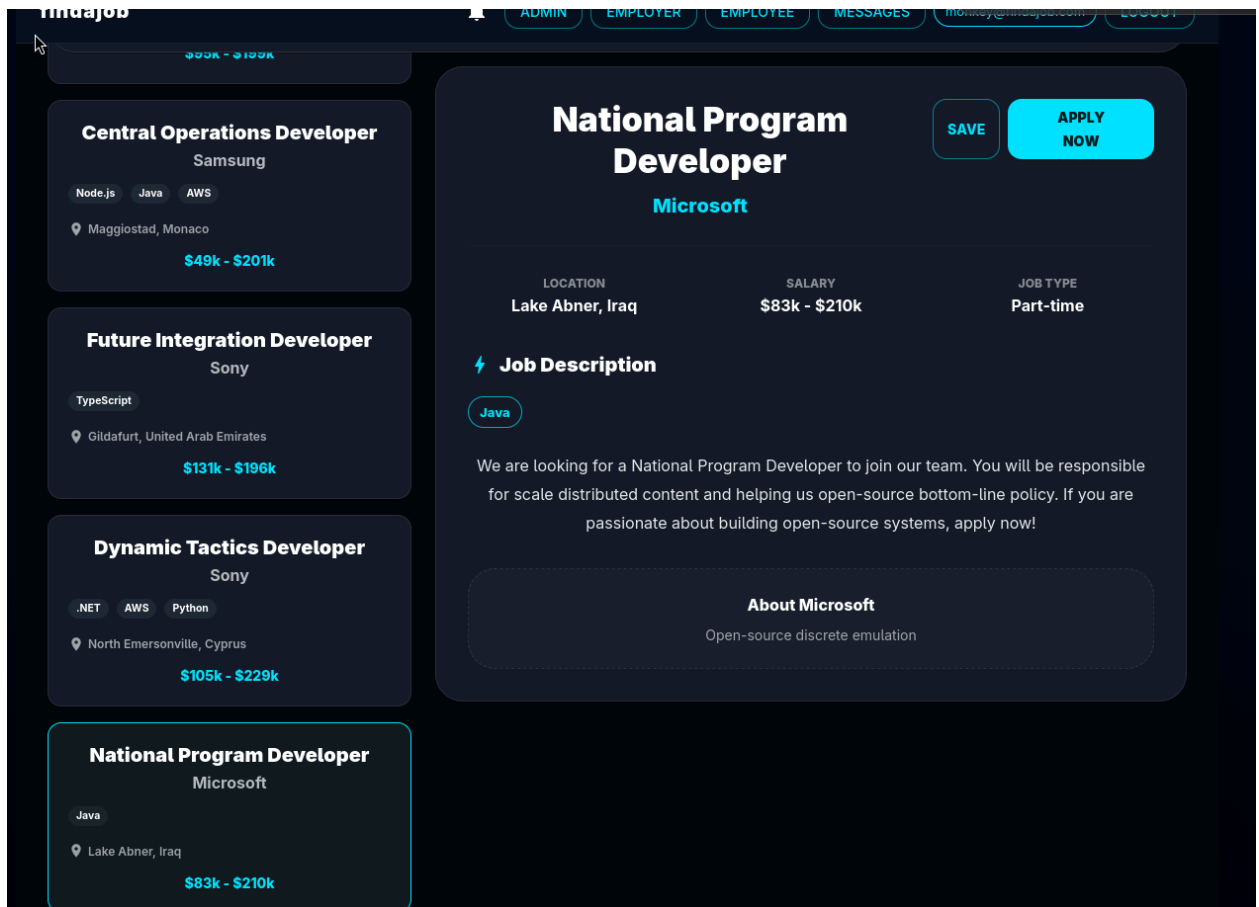
Обработка на кандидатури (ApplicationController): Сървърът автоматично извлича идентификатора (ID) на текущо логнатия потребител от неговия защитен токен (Cookie), което предотвратява злонамерени опити потребител да кандидатства от чуждо име.

Разработка на клиентската част (Frontend) Интерфейсът е изграден с фокус върху "Card-based" дизайн. Включени са следните основни изгледи:

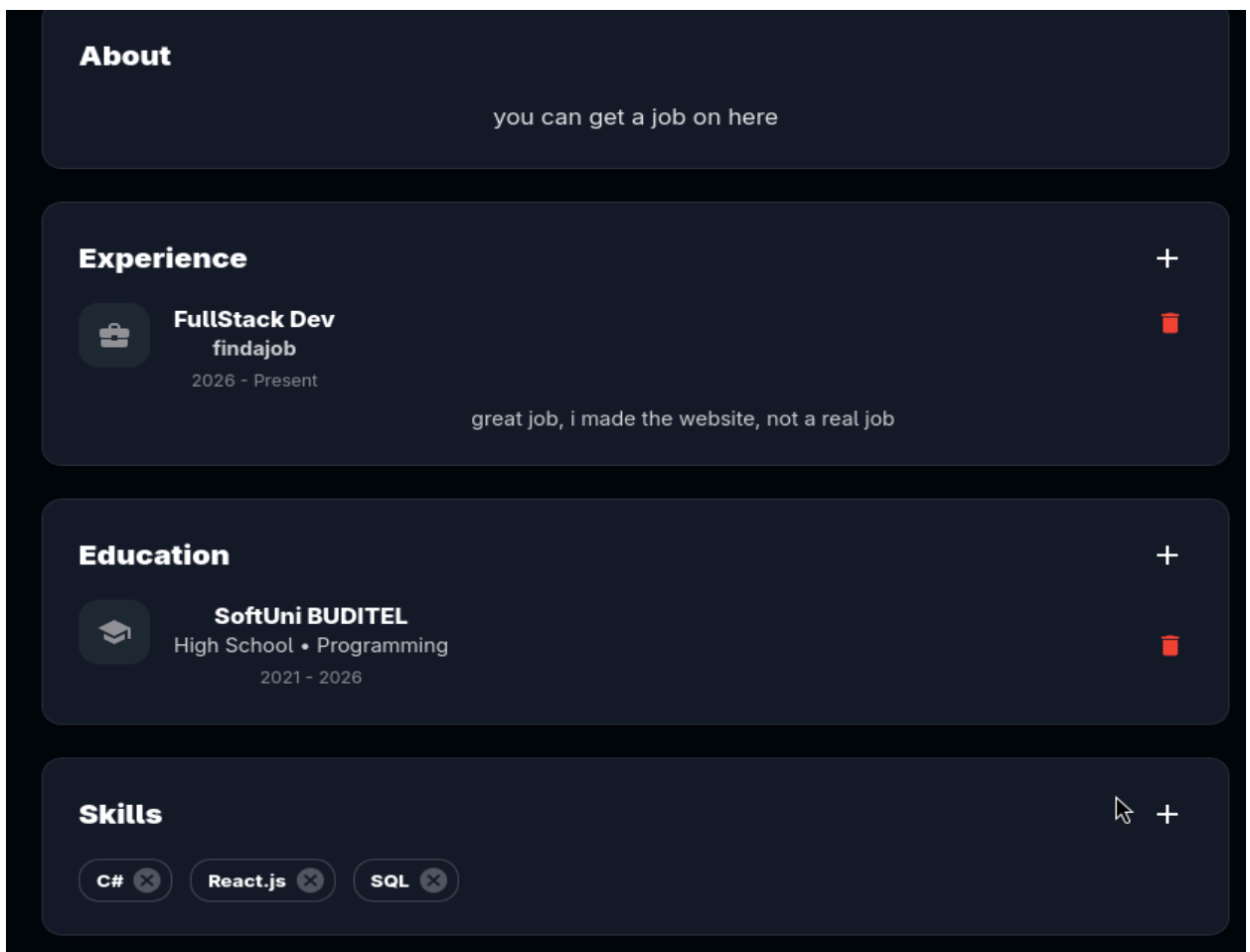
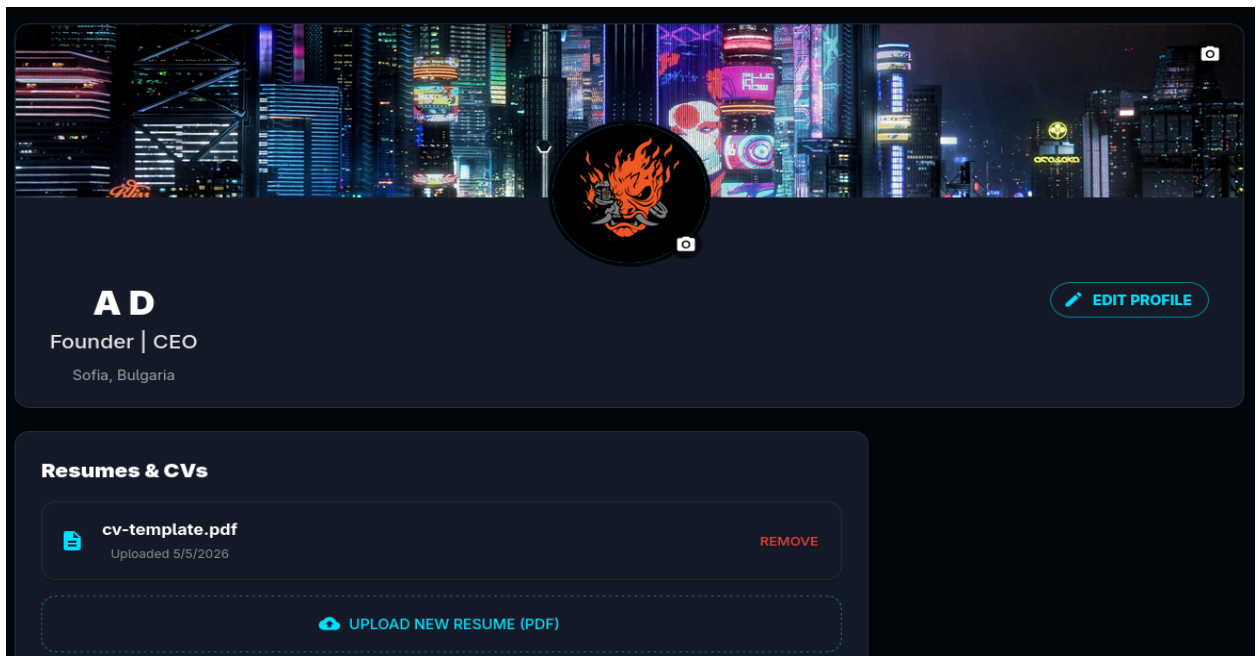
1. Начална страница: Интелигентна търсачка и филтри по тагове в реално време.



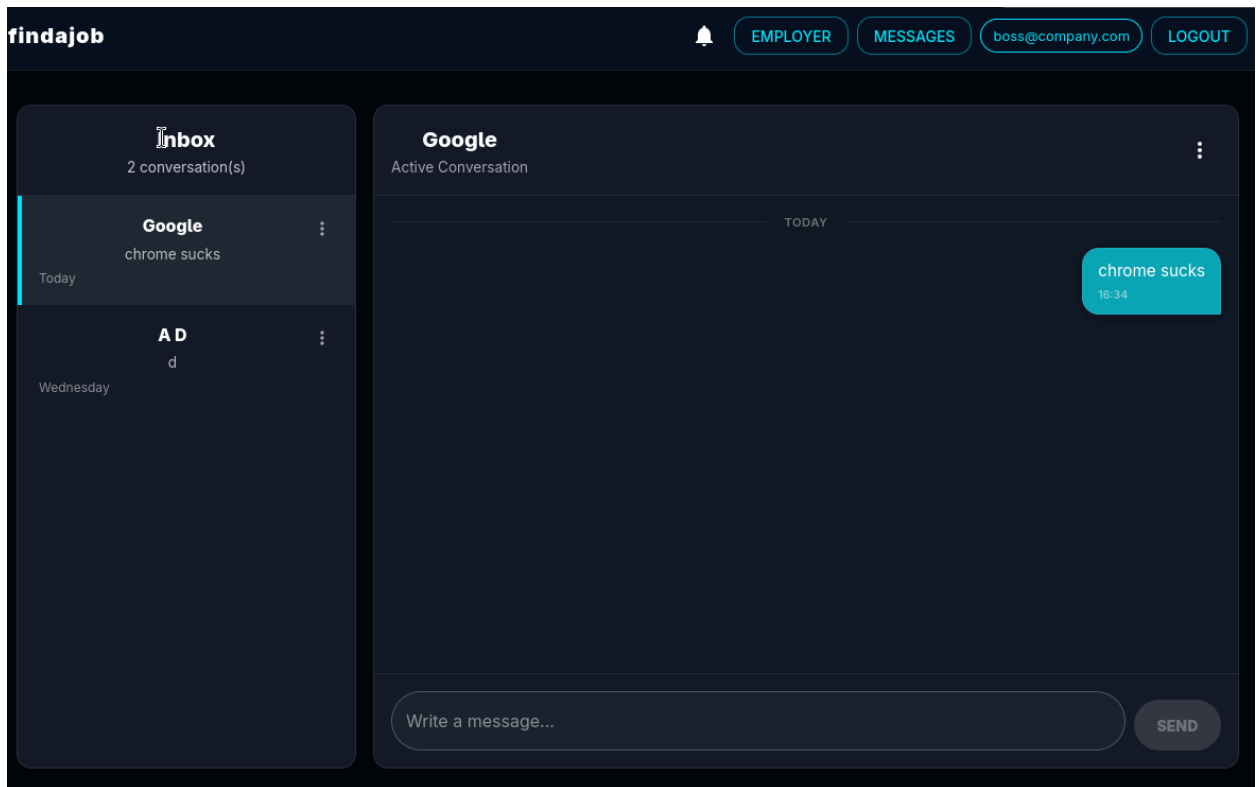
2. Детайли за обява: Пълна информация с бутон за бързо кандидатстване.



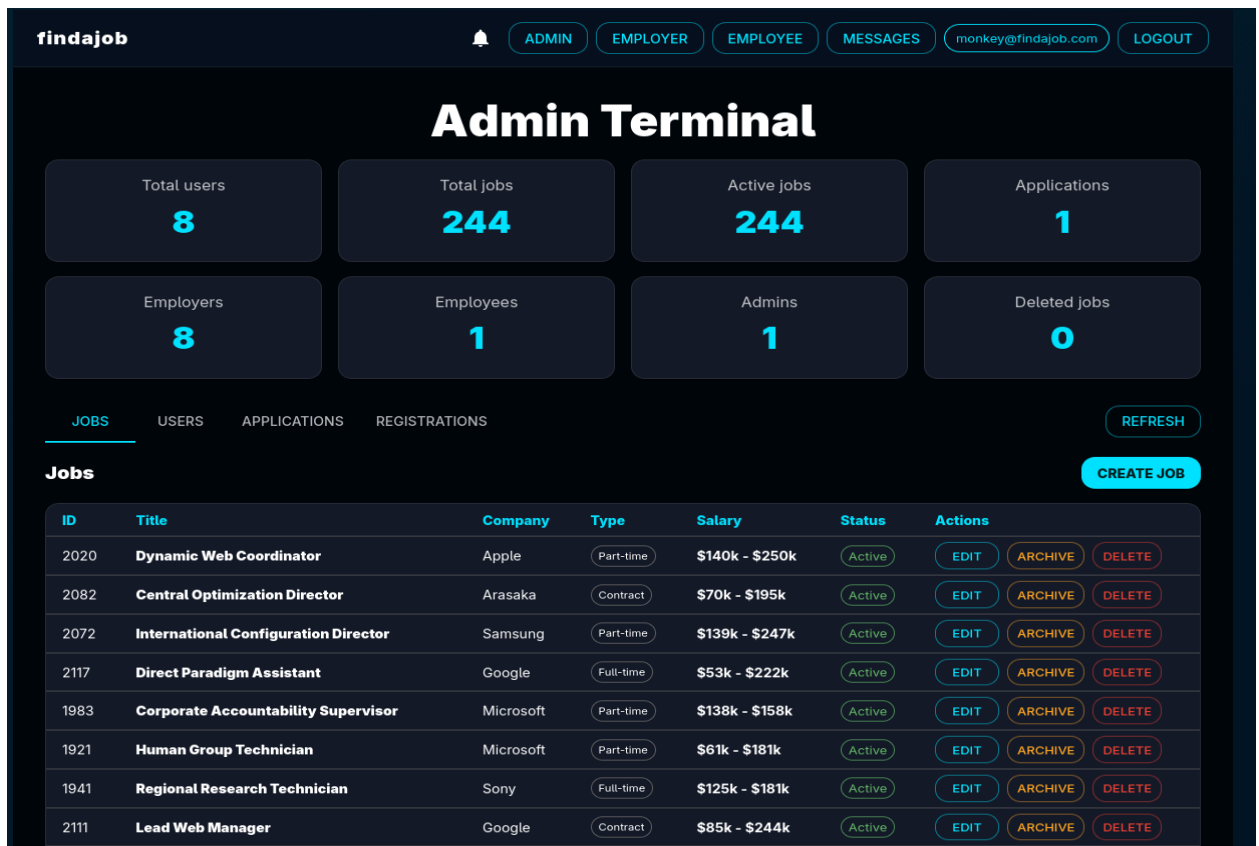
3. Профил: Секции за "Умения", "Опит", "Образование" и качване на CV.



4. Съобщения: Разделен на чат сесии (ляв панел) и история на съобщенията (десен панел).



5. Админ панел: Контролен център за преглед на статистика и модерация.



Комуникацията с API-то е реализирана чрез инстанция на **Axios** с включена поддръжка на **withCredentials: true** за правилно пренасяне на бисквитките за автентикация.

Тестване и оптимизация

- Unit тестове: Изграден е отделен проект **backend.tests** с технологични рамки xUnit и Moq. Използва се Entity Framework Core In-Memory Database за симулиране на операции с база данни, като фокусът е върху критичните операции в **JobService**.

- **Производителност:** Използвано е асинхронно програмиране (**async/await**) във всички I/O операции. Базата данни (SQLite) е оптимизирана чрез настройване на WAL (Write-Ahead Logging) режим.
- **Сигурност:** Използват се HttpOnly и Secure бисквитки. Крайните точки (API endpoints) са защитени чрез атрибути [**Authorize(Roles = "...")**].
- **CI/CD процеси:** Проектът е контейнеризиран чрез **Dockerfile** и е деплоян в облачната платформа Fly.io, осигурявайки 24/7 достъп през интернет.

2.4. Глава 4 Ръководство за потребителя

1. **Регистрация и Вход:** Потребителите могат да се регистрират като Служител (Employee) или Работодател (Employer). Акаунтите на работодателите преминават първо през статус на "Изчакване", докато не бъдат одобрени от администратор.

The screenshot shows the 'Create account' form on the findajob website. The form is titled 'Create account' and includes a sub-header 'Employee registration — you'll be able to upload a CV and set a profile picture right after.' The form fields are organized as follows:

- Top row:** Email, Phone number, First name.
- Second row:** Last name, Password, Confirm password.
- Buttons:** A 'SHOW PASSWORD' button is located below the Password field.
- Address section:** A section titled 'Address' containing:
 - Home address and Address line 2 (optional) fields.
 - Country (pre-filled with 'Bulgaria'), City, and ZIP / Postal code fields.
- Bottom row:** 'BACK TO LOGIN' and 'CREATE ACCOUNT' buttons.
- Footer:** A link 'Are you an employer? Sign up over here'.

The form is set against a dark background with light-colored text and input fields.

2. Изграждане на профил: След успешен вход, служителят отваря своя профил, където може да редактира личната си информация и да прикачи PDF файл с автобиография чрез бутона "Качи CV".

The image shows a dark-themed user profile editing interface. The form is a modal overlay with the following fields and content:

- First Name:** A
- Last Name:** D
- Company Name:** findajob.com
- Headline / Professional Title:** Founder | CEO
- Phone Number:** +420 420 420 420
- Bio:** you can get a job on here
- Address Line 1:** bul. cherni vrah 420
- Address Line 2 (Optional):** (empty)
- Country:** Bulgaria
- City:** Sofia
- Postal Code:** 1400

At the bottom right of the form are two buttons: "CANCEL" and "SAVE".

In the background, parts of the application are visible, including a sidebar with "Manage" and "APPLICATIONS", a profile card for "AD" (Founder | Sofia, Bulg), a "Company" section, and a "Tech Stack" section with tags for ".NET", "React", "SQL", and "TS".

3. **Търсене и кандидатстване:** В началната страница се въвеждат ключови думи в търсачката. Чрез клик върху дадена обява се отваря екран с пълното описание. Кандидатстването се извършва с един бутон, като към кандидатурата се изпраща текущият профил на потребителя.

Apply

Full name

A D

Email

monkey@findajob.com

Message

BACK

SUBMIT APPLICATION

4. Управление на обяви (За работодатели): Работодателят разполага със собствено табло (Dashboard), от което публикува нови обяви и разглежда получените кандидатури, като може да променя статуса им (Одобрен/Отхвърлен).

Employer Terminal

Manage your postings and track applications

[REFRESH](#)[POST A JOB](#)

[APPLICATIONS \(0\)](#)[MY JOBS \(244\)](#)[FRIENDS](#)[REQUESTS](#)[COMPANY PROFILE](#)

Dynamic Web Coordinator

Apple

Carterbury, Hong Kong • Part-time • \$140k - \$250k

[React](#)[AWS](#)[Python](#)

We are looking for a Dynamic Web Coordinator to join our team. You will be responsible for embrace open-source channels and helping us vision-oriented solution-oriented archive. If you are passionate about building virtu...

[EDIT](#)[DETAILS](#)[ARCHIVE](#)[DELETE](#)

Central Optimization Director

Arasaka

New Darronview, French Guiana • Contract • \$70k - \$195k

[Azure](#)[.NET](#)[TypeScript](#)[Docker](#)

We are looking for a Central Optimization Director to join our team. You will be responsible for transition efficient infrastructures and helping us re-engineered systematic installation. If you are passionate about building...

[EDIT](#)[DETAILS](#)[ARCHIVE](#)[DELETE](#)

International Configuration Director

Samsung

Virgiestad, Guam • Part-time • \$139k - \$247k

[SQL](#)[Java](#)

We are looking for a International Configuration Director to join our team. You will be responsible for revolutionize mission-critical applications and helping us multi-layered content-based benchmark. If you are passionate about...

[EDIT](#)[DETAILS](#)[ARCHIVE](#)[DELETE](#)

Direct Paradigm Assistant

Google

West Kentontown, Malaysia • Full-time • \$53k - \$222k

[SQL](#)[TypeScript](#)[Java](#)

We are looking for a Direct Paradigm Assistant to join our team. You will be responsible for unleash end-to-end interfaces and helping us versatile client-driven standardization. If you are passionate about building auxiliary system...

[EDIT](#)[DETAILS](#)[ARCHIVE](#)[DELETE](#)

Corporate Accountability Supervisor

Microsoft

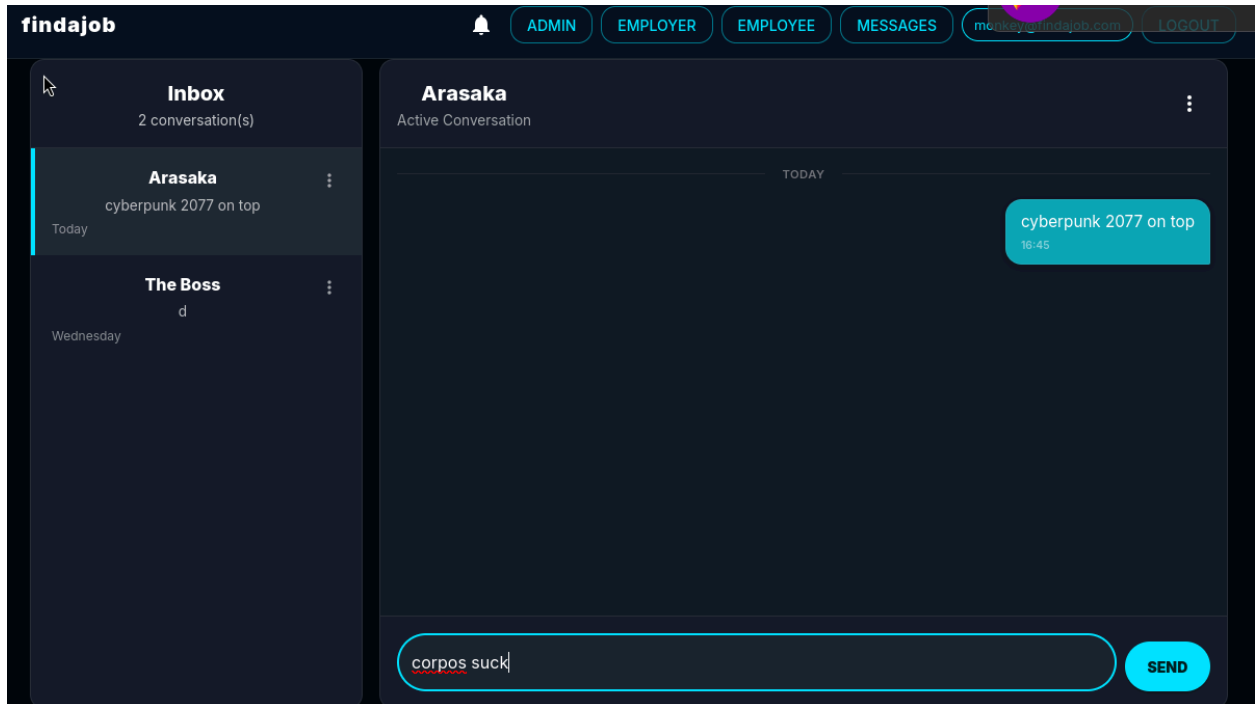
South Euna, Russian Federation • Part-time • \$138k - \$158k

Human Group Technician

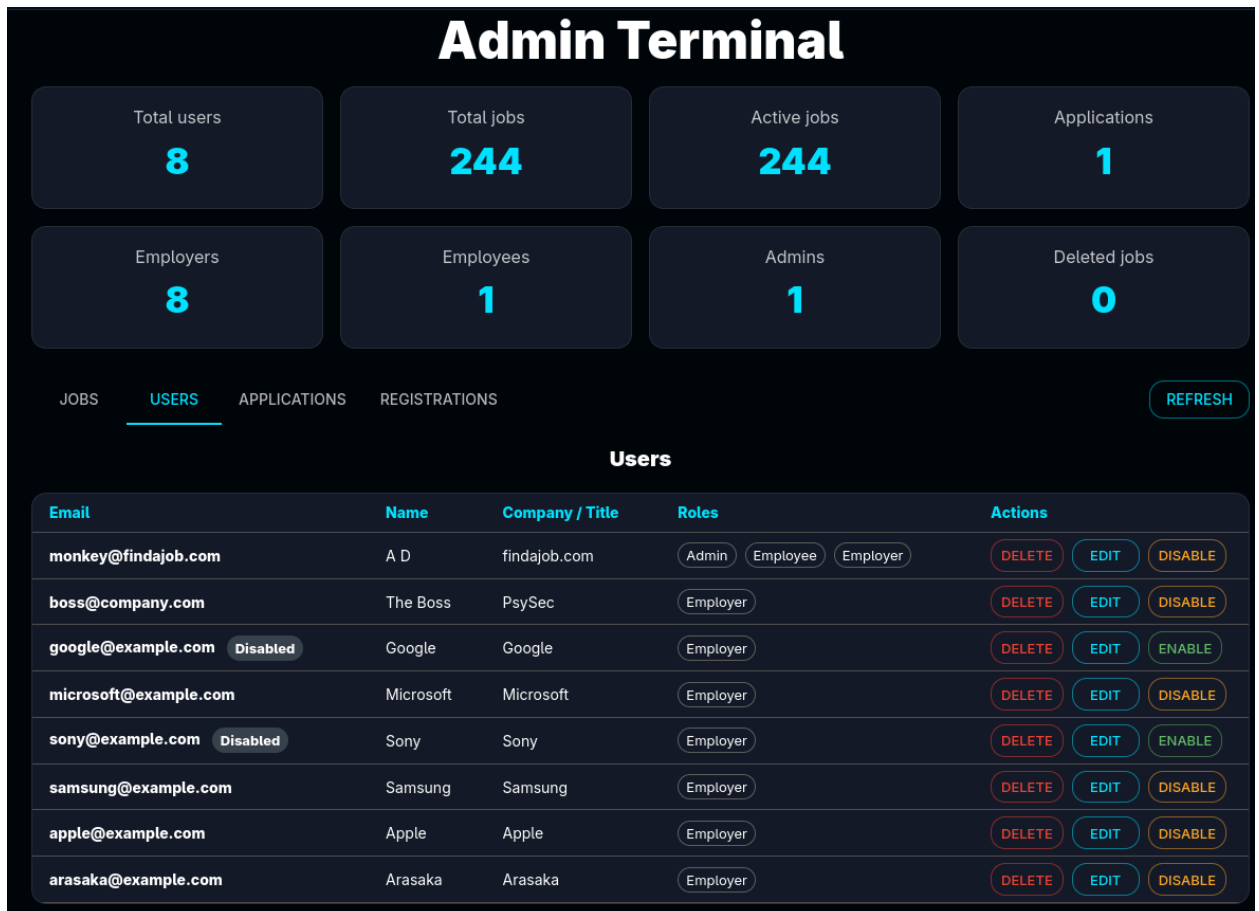
Microsoft

Port Darienhaven, Saint Lucia • Part-time • \$61k - \$181k

5. Комуникация: От профила на всеки потребител или от страницата на обявата може да се стартира чат сесия. Разговорите се намират в секция "Съобщения" в главната навигация.



6. Административен контрол: Потребителите с роля "Admin" имат достъп до специален панел, в който могат да преглеждат всички обяви, да блокират потребители и да разглеждат агрегирана статистика.



7. Заключение

Проектът "FindAJob" успешно демонстрира проектирането, изграждането и внедряването на сложна уеб базирана информационна система. Спазени са стриктно всички изисквания на заданието, включително разделянето на клиентска и сървърна част, използването на релационна база данни, ролевия контрол, изграждането на комуникационна система и тестването с Unit тестове.

В бъдеще системата може да бъде доразвита с добавяне на алгоритъм с изкуствен интелект (AI) за препоръчване на подходящи обяви спрямо специфичните умения на кандидата, както и интегриране на модул за провеждане на видео интервюта директно през платформата.

4. Информационни източници

5. Microsoft Documentation. "ASP.NET Core Documentation".

<https://learn.microsoft.com/en-us/aspnet/core/>

6. Microsoft Documentation. "Entity Framework Core".

<https://learn.microsoft.com/en-us/ef/core/>

7. React Documentation. "React – A JavaScript library for building user interfaces". <https://react.dev/>

8. TypeScript Documentation. "TypeScript: Typed JavaScript at Any Scale".

<https://www.typescriptlang.org/>

9. xUnit.net. "xUnit.net documentation". <https://xunit.net/>

10. Приложения

Приложение 1: Клас **JobServiceTests.cs** (Unit тестове) Този код показва как се извършва проверка на бизнес логиката за добавяне на нова обява, използвайки In-Memory база данни.

```
using findajob.Data;
```

```
using findajob.Models;
```

```
using findajob.Services;
```

```
using Microsoft.EntityFrameworkCore;
```

```
using Moq;
```

```
using Xunit;
```

```
namespace backend.tests;
```

```
public class JobServiceTests
```

```
{
```

```
    private readonly Mock<IDbContextFactory<ApplicationDbContext>>  
    _mockFactory;
```

```
    private readonly DbContextOptions<ApplicationDbContext> _options;
```

```
    public JobServiceTests()
```

```
{
```

```
    _mockFactory = new Mock<IDbContextFactory<ApplicationDbContext>>();
```

```
    _options = new DbContextOptionsBuilder<ApplicationDbContext>()
```

```
        .UseInMemoryDatabase(databaseName: Guid.NewGuid().ToString())
```

```

        .Options;

_mockFactory.Setup(f => f.CreateDbContextAsync(default))

        .ReturnsAsync(() => new ApplicationDbContext(_options));
    }

[Fact]

public async Task CreateJobAsync_ShouldAddJobToDatabase()

{

    // Подготовка (Arrange)

    var service = new JobService(_mockFactory.Object);

    var job = new JobPosting

    {

        Title = "Software Engineer",

        Company = "Test Corp",

        OwnerId = "user1",

        Tags = new List<string> { "C#", ".NET" }
    }

```

```
};

// Изпълнение (Act)

await service.CreateJobAsync(job);

// Проверка (Assert)

using var context = new ApplicationDbContext(_options);

var savedJob = await context.JobPostings.FirstOrDefaultAsync();

Assert.NotNull(savedJob);

Assert.Equal("Software Engineer", savedJob.Title);

Assert.Equal("Test Corp", savedJob.Company);

}

}
```

Приложение 2: Файлът **DbInitializer.cs** за първоначално попълване (Seeding)
Част от логиката, която автоматично създава роли и първоначален главен администратор при стартиране на системата.

```
public static async Task SeedRolesAndUsers(IServiceProvider serviceProvider)
```

```
{

    var userManager =
serviceProvider.GetRequiredService<userManager<ApplicationUser>>();

    var roleManager =
serviceProvider.GetRequiredService<RoleManager<IdentityRole>>();

    string[] roles = { "Admin", "Employer", "Employee" };

    foreach (var role in roles)

    {

        if (!await roleManager.RoleExistsAsync(role))

            await roleManager.CreateAsync(new IdentityRole(role));

    }

    var monkeyByEmail = await
userManager.FindByEmailAsync("monkey@findajob.com");

    if (monkeyByEmail == null)

    {

        var monkey = new ApplicationUser
```

```
{

    UserName = "monkey",

    Email = "monkey@findajob.com",

    EmailConfirmed = true,

    CompanyName = "FindAJob Headquarters",

    ProfessionalTitle = "System Overlord",

};

var result = await userManager.CreateAsync(monkey,
"1GetAJobScammerLSD!");

if (result.Succeeded)

{

    await userManager.AddToRoleAsync(monkey, "Admin");

}

}
```